

A DoMS Bootstrap

Before initializing a DoMS instance, the SDK builds a logical representation (referred as DoMSMeta) of the DoMS that can be managed by MODB at runtime based on the DoMS specification provided by the user.

DoMSMeta is a set of objects that stores metadata about the data model, data constraints, and the mappings of events to application functions. These are leveraged to check the correctness of intra-DoMS properties, exploring the specification space to identify anomalies related to functions that append to its own input event log, event logs with multiple functions appending into it, functions without input event log, output events without associated event log, writes to foreign data items, writes to and reads from non-declared tables, queries that include non-existing fields, functions with more than an input event log, missing data dependence (i.e., incorrect foreign key declaration), and more.

These anomalies are detected by navigating through the abstract syntax tree (AST) of user-defined classes and correlating information extracted from the meta-programming constructs, e.g., annotations. If an anomaly is detected, the DoMS initialization procedure is stopped and the user is informed about the model violations.

Afterwards, MODB SDK partners with JVM bootstrap class loader to preload and inject system functionality into key user classes in the execution path of DoMS functions so to make their lifecycle and MODB internals transparent to developers. For instance, it is not a goal to expose how an output event is appended to an event log nor to disclose internal index structures used to store application state.

Specifically, concrete implementations of @Repository interfaces are created via byte code generation and the generated code is instrumented to intercept database operations and forward them to an internal transaction management layer (§ 6). @Repository custom instances are then injected into preloaded @Microservice instances. Besides, complex queries (e.g., with aggregate, join, and where clauses) declared as part of @Repository interfaces are parsed and have their query execution plans generated in advance to speed up DoMS execution.

Apart from managing application entities that map to database rows, necessary in every ORM-based application, no other objects require being managed by users. The MODB SDK hides system complexities entirely, thereby allowing for clean application logic specification.

Besides intra-DoMS properties, inter-DoMS properties also need to be checked for correctness. In particular, a single DoMS instance appender per event log must hold, event schemas must match across DoMS instances, foreign tables from every DoMS must map to a single native table from another DoMS unique DoMS name identifiers, correctness of transaction definitions, and others.

After initialization, each DoMS instance forwards its DoMSMeta to the Catalogs, which are then combined for correctness check of inter-DoMS properties by the dispatchers. If some property is violated, the DoMS instances involved are not allowed to participate in multi-DoMS transactions. Afterward, the MODB bootstrap process is considered completed.

B Algorithms

B.1 Extended Scheduling Algorithm

Based on Algorithm 2, we divide transaction scheduling in a DoMS into three functions, *Ingest*, *Schedule*, and *TxnCompletionCallback*, which are executed by different threads concurrently.

The *Ingest* prepares transaction inputs for scheduling by reading all input events available in the inbound event stream and building transaction contexts for execution. A transaction context maps an input event to the respective application function of a DoMS. The *Schedule* tracks the execution of transactions continuously, making a best effort to schedule the largest number of concurrent transactions possible, respecting the global order. Like the *Ingest*, it blocks at specific points to release resources for transaction execution. The *TxnCompletionCallback* is executed by the thread assigned to execute a transaction. It updates the *lastTidExec* and the sets *partSched* (set of partitioned transactions scheduled) and *paraSched* (set of parallel transactions scheduled) atomically to allow for the scheduling thread to progress with further transaction scheduling. In the event of an error, the abort procedure is triggered, as covered in Algorithm 3.

B.2 Transaction Abort Algorithm

Algorithm 3 presents the abort procedure.

C Scheduling Correctness

In this section, we analyze the correctness of the DoMS scheduling algorithm. It is based on the assumption that users provide a correct specification of application function execution modes.

THEOREM C.1. *Let $O = [T_1, T_2, \dots, T_n]$ be a total order of transactions defined by the dispatchers, and S be the schedule produced by the DoMS. S is conflict-equivalent to O .*

PROOF. The DoMS scheduling algorithm ensures that transactions with different execution modes {Single threaded, Partitioned, Parallel} are not scheduled concurrently. It puts transactions into a sequence of groups, where all the transactions in a group belong to exactly one of the three execution modes. Transactions in each group have TiDs that are greater than all the TiDs of the transactions in any preceding groups. Furthermore, transactions in each group are completely executed before all the transactions in the subsequent groups, except for read-only (R) transactions. The algorithm ensures that all the R transactions are dispatched after all the potentially conflicting RW transactions with smaller TiDs are executed. Therefore, the multi-version database can ensure the latter reads the version according to the schedule stated in O . In the following analysis, we only focus on RW transactions.

For each transaction group, there are three possible cases:

Case (i). All transactions are single-threaded transactions. The DoMS scheduling algorithm ensures that for any pair of single-threaded RW transactions, they are executed strictly according to the order in O without concurrency.

Case (ii). All transactions are partitioned transactions. Let $G(O)$ be the conflict graph of the transactions in this group induced by the total order O , and $G(S)$ be the corresponding conflict graph induced by the schedule S .

Let $T_i \rightarrow T_j \in G(O)$. This implies:

- T_i and T_j conflict.

Algorithm 2: DoMS transaction scheduling

Input: Infinite input event stream $I = (e_1, e_2, \dots)$
Input: $lastTidCommitted$ (0 by default)
Output: Infinite output event stream $O = (o_1, o_2, \dots)$

```

1  $txns \leftarrow \emptyset$ ; // transaction context set
2  $lastTidExec \leftarrow lastTidCommitted$ ;
3  $paraSched \leftarrow \emptyset$ ; // parallel txns scheduled
4  $partSched \leftarrow \emptyset$ ; // partitioned txns scheduled

5 Function Schedule():
6    $txn \leftarrow txn' \in txns : txn'.prevTid = lastTidExec$ ;
7   if  $txn.mode = SINGLE$  then
8     if  $\neg(partSched \neq \emptyset \vee paraSched \neq \emptyset) \vee (\exists t : t \in$ 
9        $partSched \cup paraSched \wedge t.type = RW)$  then
10      block while above cond. holds;
11      if  $t.type = R$  then  $dispatch(t)$ ; else  $execute(t)$ ;
12      goto line 6;
13   do
14     if  $txn.mode = PARA \wedge (partSched = \emptyset \vee (\forall t : t \in$ 
15        $partSched \wedge t.type = R))$  then
16        $paraSched \leftarrow paraSched \cup txn$ ;
17        $dispatch(txn)$ ;
18     else if  $txn.mode = PART \wedge (paraSched = \emptyset \vee (\forall t : t \in$ 
19        $paraSched \wedge t.type = R))$  then
20       if  $partSched[t.part] = \emptyset \vee (\forall t : t \in$ 
21          $partSched[t.part] \wedge t.type = R)$  then
22          $partSched \leftarrow partSched \cup txn$ ;
23          $dispatch(txn)$ ;
24       else block until  $partSched[t.part] = \emptyset$ ;
25     else
26       goto line 6;
27    $nextTxn \leftarrow txn' \in txns : txn'.prevTid = txn.Tid$ ;
28    $execModeAux \leftarrow txn.mode$ ;
29    $txn \leftarrow nextTxn$ ;
30   while  $txn \neq \emptyset \wedge execModeAux = txn.mode$ ;

```

```

27 Function Ingest():
28   while true do
29      $I' \leftarrow pollAllBlocking(I)$  where  $I' \subset I$ ;
30     for  $e \in I'$  do
31        $f \leftarrow mapFunction(e)$ ;
32        $txnCtx \leftarrow buildTxnCtx(e, f)$ ;
33        $txns \leftarrow txns \cup txnCtx$ 

34 Function TxnCompletionCallback( $txn, status$ ):
35   if  $status = ERROR$  then
36      $abort(txn)$ ;
37     return;
38    $lastTidExec \leftarrow max(lastTidExec, txn.Tid)$ ;
39    $txn.isFinished \leftarrow true$ ;
40    $O \leftarrow O \cup txn.result$ ;
41   if  $txn.mode = PARA$  then
42      $paraSched \leftarrow paraSched \setminus txn$ ;
43   else if  $txn.mode = PART$  then
44      $partSched \leftarrow partSched \setminus txn$ ;

```

- $T_i \prec T_j$ in O .

The algorithm ensures that RW transactions accessing conflicting partitions are never scheduled concurrently. Therefore, for any pair of conflicting transactions T_i and T_j such that $T_i \prec T_j$ in O , the conflicting operations from T_i precede those from T_j in S . Hence, the edge $T_i \rightarrow T_j$ exists in the conflict graph $G(S)$, preserving the conflict-equivalence with $G(O)$.

Case (iii). All the transactions are embarrassingly parallel transactions. As we assume that they have no conflicting access to the DoMS state, $G(O)$ of the transactions in this group does not have any edges. Therefore, the corresponding $G(S)$ does not violate any precedence relationship in $G(O)$.

All in all, the complete schedule S is conflict-equivalent to the schedule specified by O . $\square$

D Implementation and Optimizations

To provide a high-performance cross-stack DBMS **that unifies event and state management**, numerous optimizations are required, including network I/O, transaction scheduling, concurrent data access, query processing, and storage formats for both data and event logs.

Network I/O Processing. Network communication with the dispatcher and among DoMSes is achieved with TCP connections through asynchronous I/O [64]. This applies to both event logs and protocol messages. I/O threads are divided into two categories, receivers and senders. The OS kernel signals receivers about new network I/O events, which in turn unmarshal log entries from the buffers, deserialize individual input events, and queue them for transaction processing. Whenever output events are made available, senders dispatch them as log segments to the corresponding consumer DoMSes. Note that **there can be multiple consumers for an event**. If there is a failure to send (e.g., a DoMS crash or a

Algorithm 3: DoMS transaction abort

Input: txn that led to a conflict or constraint violation
Input: Infinite input event stream $I = (e_1, e_2, \dots)$
Input: Transaction context set $txns$
Input: Set of DoMS tables T

```

1 block ( $I$ ); // block Ingest polling  $I$ 
2 block ( $txns$ ); // block Schedule access to  $txns$ 
3 if ( $txn$  abort is raised by this DoMS) then
4   forward abort msg to COMMIT_HANDLER;
5    $txnsToAbort \leftarrow \forall txn' \in txns : txn'.Tid \geq txn.Tid;$ 
6    $txns \leftarrow txns \setminus txnsToAbort;$ 
7   for  $table \in T$  do
8     for  $txnToAbort \in txnsToAbort$  do
9        $table.versions \leftarrow$ 
10         $table.versions \setminus txnToAbort.writeSet;$ 
11 if (DoMS is the first node in the transaction graph) then
12   log  $txn$  abort event in DoMS state;
13   find the successor and predecessor events of  $txn$  aborted
14   in  $I$ ;
15   adjust dependencies;
16   reinsert events after  $txn$  aborted in  $I$ ;
17 else
18   find the predecessor event of  $txn$  aborted in  $I$ ;
19   wait until successor event of the predecessor is sent
20   through  $I$ ;
21  $lastTidExec \leftarrow predecessor.Tid;$ 
22 unblock ( $I$ );
23 unblock ( $txns$ );

```

network partition), log segments are placed in a pending buffer for later reattempt.

A configurable number of receivers are placed in a network thread pool and return when no network events are available. Senders run independently and are uniquely assigned a DoMS consumer to send log entries to. This prevents cache pollution, i.e., cache lines of network events evicting cache lines of the DoMS consumer, and vice versa. In this case, when output event logs are unavailable, senders yield their CPU time to other system threads.

Transaction Dispatching. Network I/O processing in dispatchers follows a design similar to a DoMS but is targeted at maximizing transaction request dispatch while not jeopardizing the TiD assignment process. To meet this goal, sender I/O threads never block. Instead, in the absence of new event inputs, they yield their CPU time to another thread. This minimizes waiting times for DoMSes because blocking would cause the I/O thread to park and unpark, a costly process for input generation. Receiver I/O threads function in the same way as in a DoMS. Dispatchers assign TiDs to multi-DoMS transactions and compute the dependencies of each DoMS involved. They operate as independent threads in an event loop mode (i.e., non-blocking), outside the context of a thread pool.

E High Availability

DoMS. The DoMSes can replicate their produced event logs to standby replicas for failure recovery. In this case, sender I/O threads,

in addition to logging log segments, can stream them to replicas. The replicas process the log segments in the same way as the primary, but do not send output events to consumer DoMS. For failure detection, replicas rely on heartbeats received from primary nodes. Upon detection of a failure, replicas trigger a leader election. Similar to Raft [63], replicas start as candidates and rely on the number of votes they receive to determine whether their candidacy succeeds. In this case, it broadcasts the election result. As transactions execute optimistically (§ 5.1), a newly elected leader may not contain some log segments. Therefore, it is necessary to request possible missing log segments from upstream DoMS. Note that the Catalogs must also be checked for abort information of in-progress batches. In this case, replicas must proceed in the same way as the primary to safeguard correctness. In other words, Algo. 3 must also be executed. Thereafter, the system can return to normal operation.

Proxies. Proxies act as stateless load balancers. In case of failures or newly deployed proxies, they obtain from the Catalogs the latest DoMS metadata and transaction definition versions stored. This procedure repeats if an invocation fails due to a deprecated DoMS metadata or transaction definition.

Dispatchers and Commit Handlers.

Upon all *batch_complete* messages' arrival, a commit handler logs a *batch_commit* message to the Catalogs. Upon a crash, a standby replica requests the last committed batch ID from all DoMSes to cover cases where the commit handler crashes before logging it. Upon receiving the responses, it updates its last committed batch ID if necessary and sends a *batch_aborted* message to all DoMSes to abort all ongoing batches at the time of the crash. DoMSes then proceed similarly to a transaction abort. DoMSes discard non-committed data item versions, and TiD precedences are adjusted accordingly to receive transactions from the new process.

Differently from DoMSes the dispatchers do not stream event logs to replicas since these are simply transaction requests that can be retried by clients. Although this may lead to losing some useful work the DoMSes performed, it avoids a TCP round trip to each replica for every transaction emitted.

F DoMS Execution Modes

Table 2 exhibits the DoMS execution modes used in Online Marketplace and Table 3 exhibits for TPC-C. In Dapr, all operations run as a parallel task given the lack of transaction isolation.

G PostgreSQL Tuning

In the application ORM, we set the maximum number of connections to 10K, bypassing the default maximum of 100 connections, and we configure a connection pool to reuse connections. Besides, we disable synchronous commit, so updates are eventually logged rather than atomically¹. We use Unix domain socket for inter-process, kernel-based communication instead of the default TCP sockets. We increase the `pgsql` buffer size to improve analytical query performance. We increase the shared buffer from 128MB to 3GB, which corresponds to 15% of 21 GB available in `c5n.2xlarge`.

¹In off mode, there is no waiting, so there can be a delay between when success is reported to the client and when the transaction is guaranteed to be safe against a server crash (the maximum delay is $3X \text{ wal_writer_delay}$).

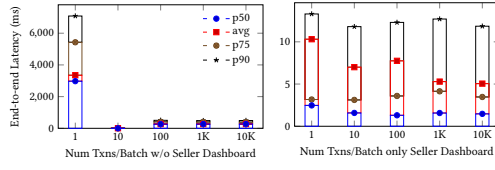


Figure 10: Effect of Batch Size on Latency

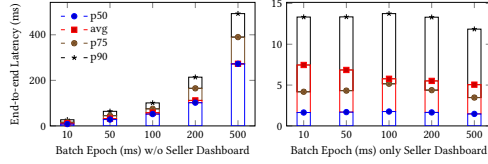


Figure 11: Effect of Batch Epoch on Latency

Table 2: DoMS execution modes in Online Marketplace

Component	Transaction	Primitive
Cart	Customer Checkout	Keyed on customer_id
Cart	Price Update	Single thread; can't extract from the event which carts contain the product
Stock	Customer Checkout	Runs single-threaded in the experiments but can be keyed on seller_id, product_id
Product, Cart, Stock	Product Delete	Keyed on seller_id, product_id
Product	Price Update	Keyed on seller_id, product_id
Order	Customer Checkout	Keyed on customer_id
Payment	Customer Checkout	Parallel
Shipment	Customer Checkout	Parallel
Seller	Seller Dashboard	Parallel

Table 3: DoMS execution modes in TPC-C

Component	New Order	Payment	Order Status
Warehouse	Keyed on warehouse and district IDs	Keyed on warehouse and district IDs	Read-only, thus parallel
Inventory	Runs keyed on supplier warehouse IDs in experiments but can be keyed on item IDs for increased concurrency	-	-
Order	Parallel	Parallel	Read-only, thus parallel

instance, as suggested by PostgreSQL documentation. The work mem is set to 128 MB instead of the default 4MB.

H Additional Experiments

H.1 Effect of Batch Size

To study the effects of the number of transactions per batch in OM, we start by fixing the batch epoch to infinity, varying the number of transactions in a batch, and maximizing the concurrency level. In Figure 9 (left), we observe an average of 1K tx/sec increase up to batch size 1K, the point where throughput stabilizes. Next, we fix the maximum number of transactions in a batch to infinity and vary the epoch. Figure 9 (right) demonstrates the effectiveness of the commit protocol, showing little variation across epochs.

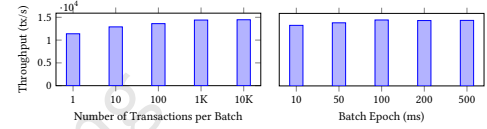


Figure 9: Effect of Batch Size on Throughput

Figure 10 (left) shows the substantial overhead of network I/O if using a single transaction per batch. It is worth noting that 10 transactions per batch achieves low latency and doubles Dapr's throughput, providing a good trade-off in this particular scenario. Besides, Figure 10 (right) shows the insensitivity of Seller Dashboard's latency to the batch sizes. On the other hand, Figure 11 shows that varying the batch epoch leads to a more natural variation in latency for transactions committing in a batch, without affecting throughput though.

H.2 Effect of Scale Factor on Latency

Figure 12 reports the statistics of the intervals between the commit of each pair of adjacent batches for each scale factor in TPC-C. In MODB, batch latencies decrease because batches are filled and processed more quickly due to the higher number of requests. This effect is also observable in MODB with logging and checkpointing with an average of 17% overhead due to the intensive I/O incurred by the durable write procedures.

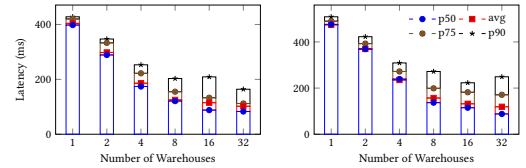


Figure 12: Effect of Scale Factor on Batch Latency in TPC-C. In-memory execution (left) and with durability (right)

I MODB TPC-C Implementation

```

1 @Inbound(values = "new-order-ware-in")
2 @Outbound("new-order-ware-out")
3 @Transactional(type = RW)
4 @PartitionBy(clazz = NewOrderWareIn.class, method = "getId")
5 public NewOrderWareOut processNewOrder(NewOrderWareIn in) {
6     District district = this.districtDB.lookupByKey(new
7         District.DistrictId(in.d_id, in.w_id));
8     float w_tax = this.warehouseDB.getWarehouseTax(in.w_id);
9     district.d_next_o_id++;
10    this.districtDB.update(district);
11    float c_discount = this.customerDB.getDiscount(in.c_id, in.d_id,
12        in.w_id);

```



```

2089 11 return new NewOrderWareOut(in.w_id, in.d_id, in.c_id,
2090      in.itemsIds, in.supWares, in.qty, in.allLocal, w_tax,
2091      district.d_next_o_id, district.d_tax, c_discount);
2092 12 }
2093 13 @Inbound(values = "payment-in")
2094 14 @Outbound("payment-out")
2095 15 @Transactional(type = RW)
2096 16 @PartitionBy(clazz = PaymentIn.class, method = "getId")
2097 17 public PaymentOut processPayment(PaymentIn in) {
2098 18     District district = this.districtDB.lookupByKey(new
2099         District.DistrictId(in.d_id, in.w_id));
2100 19     district.d_ytd += in.amount;
2101 20     Warehouse warehouse = this.warehouseDB.lookupByKey(in.w_id);
2102 21     warehouse.w_ytd += in.amount;
2103 22     this.districtDB.update(district);
2104 23     this.warehouseDB.update(warehouse);
2105 24     Customer customer;
2106 25     if(in.by_name){
2107 26         List<Customer> customers =
2108             this.customerDB.getCustomerByLastName(in.c_d_id,
2109                 in.c_w_id, in.c_last);
2110 27         int index = customers.size() / 2;
2111 28         if (customers.size() % 2 == 0) index -= 1;
2112 29         customer = customers.get(index);
2113 30     } else {
2114 31         customer = this.customerDB.lookupByKey(new
2115             Customer.CustomerId(in.c_id, in.d_id, in.w_id));
2116 32     }
2117 33     if (customer.c_credit.equals("BC")) {
2118 34         customer.c_data = "%d %d %d %d %d %f |
2119             %s".formatted(customer.c_id, in.c_d_id, in.c_w_id,
2120                 in.d_id, in.w_id, in.amount, customer.c_data);
2121 35         if (customer.c_data.length() > 500) {
2122 36             customer.c_data = customer.c_data.substring(0, 500);
2123 37         }
2124 38     }
2125 39     customer.c_balance -= in.amount;
2126 40     customer.c_ytd_payment += in.amount;
2127 41     customer.c_payment_cnt += 1;
2128 42     this.customerDB.update(customer);
2129 43     String h_data = "%s %s".formatted(warehouse.w_name.length() >
2130         10 ? warehouse.w_name.substring(0, 10) : warehouse.w_name,
2131         district.d_name.length() > 10 ?
2132         district.d_name.substring(0, 10) : district.d_name);
2133 44     return new PaymentOut(in.w_id, in.d_id, in.c_id, in.c_w_id,
2134         in.c_d_id, in.amount, h_data);
2135 45 }
2136 46 @Inbound(values = "order-status-in")
2137 47 @Outbound("order-status-out")
2138 48 @Transactional(type = R)
2139 49 public OrderStatusOut processOrderStatus(OrderStatusIn in) {
2140 50     if(in.by_name){
2141 51         List<Customer> customers =
2142             this.customerDB.getCustomerByLastName(in.d_id,
2143                 in.w_id, in.c_last);
2144 52     } else {
2145 53         Customer customer = this.customerDB.lookupByKey(new
2146             Customer.CustomerId(in.c_id, in.d_id, in.w_id));
2147 54     }
2148 55     return new OrderStatusOut(in.w_id, in.d_id, in.c_id);
2149 56 }
2150 57 @Inbound(values = "delivery-out")
2151 58 @Transactional(type = RW)
2152 59 @PartitionBy(clazz = DeliveryOut.class, method = "getId")
2153 60 public void processDelivery(DeliveryOut in) {
2154 61     List<Customer.CustomerId> customerIds = new
2155         ArrayList<>(NUM_DIST_PER_WARE);
2156 62     for(int d_id = 1; d_id <= NUM_DIST_PER_WARE; d_id++) {
2157 63         customerIds.add(new
2158             Customer.CustomerId(in.customerIds[d_id-1], d_id,
2159                 in.w_id));
2160 64     }
2161 65     List<Customer> customers =
2162         this.customerDB.lookupByKeys(customerIds);
2163 66     int idx = 0;
2164 67     for(Customer customer : customers) {
2165 68         customer.c_balance += in.amounts[idx++];
2166 69     }
2167 70 }

```

```

70     customer.c_delivery_cnt++;
71 }
72 this.customerDB.updateAll(customers);
73 }

```

Listing 4: DoMS Warehouse

```

1 @Repository
2 public interface IStockRepository extends
3     IRepository<Stock.StockId, Stock> {
4     @Query("select distinct s_i_id from stock where s_i_id IN
5         (:itemIds) and s_w_id = :w_id and s_quantity < :threshold")
6     int[] getStockCount(int[] itemIds, int w_id, int threshold);
7 }
8 @Inbound(values = "new-order-ware-out")
9 @Outbound("new-order-inv-out")
10 @Transactional(type = RW)
11 @PartitionBy(clazz = NewOrderWareOut.class, method = "getId")
12 public NewOrderInvOut processNewOrder(NewOrderWareOut in) {
13     float[] prices = this.itemDB.getPricePerItemId(in.itemsIds);
14     String[] ol_dist_info = new String[in.itemsIds.length];
15     List<Stock> stockItemsToUpdate = new ArrayList<>(prices.length);
16     for(int i = 0; i < in.itemsIds.length; i++){
17         Stock stock = this.stockDB.lookupByKey(new
18             Stock.StockId(in.itemsIds[i], in.supWares[i]));
19         ol_dist_info[i] = stock.getDistInfo(in.d_id);
20         int ol_quantity = in.qty[i];
21         if(stock.s_quantity > ol_quantity){
22             stock.s_quantity = stock.s_quantity - ol_quantity;
23         } else {
24             stock.s_quantity = stock.s_quantity - ol_quantity + 91;
25         }
26         stock.s_ytd = stock.s_ytd + ol_quantity;
27         stock.s_order_cnt++;
28         if(stock.s_w_id != in.w_id){
29             stock.s_remote_cnt++;
30         }
31         stockItemsToUpdate.add(i, stock);
32     }
33     this.stockDB.updateAll(stockItemsToUpdate);
34     return new NewOrderInvOut(in.w_id, in.d_id, in.c_id,
35         in.itemsIds, in.supWares, in.qty, in.allLocal, in.w_tax,
36         in.d_next_o_id, in.d_tax, in.c_discount, prices,
37         ol_dist_info);
38 }
39 @Inbound(values = "stock-level-ord-out")
40 @Transactional(type = R)
41 public void processStockLevel(StockLevelOrdOut in) {
42     int threshold = randomNumber(10, 20);
43     int[] itemIds = this.stockDB.getStockCount(in.itemsIds, in.w_id,
44         threshold);
45 }

```

Listing 5: DoMS Inventory

```

1 @Inbound(values = "new-order-inv-out")
2 @Transactional(type = W)
3 @Parallel
4 public void processNewOrder(NewOrderInvOut in){
5     Order order = new Order(in.d_next_o_id, in.d_id, in.w_id,
6         in.c_id, new Date(), in.itemsIds.length, in.allLocal);
7     NewOrder newOrder = new NewOrder(in.d_next_o_id, in.d_id,
8         in.w_id);
9     this.orderDB.insert(order);
10    this.newOrderDB.insert(newOrder);
11    List<OrderLine> orderLinesToInsert = new
12        ArrayList<>(in.itemsIds.length);
13    for(int i = 0; i < in.itemsIds.length; i++){
14        float ol_amount = (in.qty[i] * in.itemsIds[i] * (1 +
15            in.w_tax + in.d_tax) * (1 - in.c_discount));
16        OrderLine orderLine = new OrderLine(in.d_next_o_id, in.d_id,
17            in.w_id, i+1, in.itemsIds[i], in.supWares[i], null,
18            in.qty[i], ol_amount, in.ol_dist_info[i]);
19        orderLinesToInsert.add(i, orderLine);
20    }
21    this.orderLineDB.insertAll(orderLinesToInsert);
22 }

```

```

2205 18 @Inbound(values = "payment-out")
2206 19 @Transactional(type = W)
2207 20 @Parallel
2208 21 public void processPayment(PaymentOut out){
2209 22     History history = new History(out.c_id, out.c_d_id, out.c_w_id,
2210 23         out.d_id, out.w_id, new Date(), out.amount, out.data);
2211 24     this.historyDB.insert(history);
2212 25 }
2213 26 @Inbound(values = "order-status-out")
2214 27 @Transactional(type = R)
2215 28 public void processOrderStatus(OrderStatusOut in){
2216 29     Order order = this.orderDB.getLastOrderByCustomerId(in.c_id);
2217 30     List<OrderLineInfoDto> orderLinesInfo =
2218 31         this.orderLineDB.getOrderLinesInfo(order.o_id,
2219 32         order.o_d_id, order.o_w_id);
2220 33 }
2221 34 @Inbound(values = "stock-level-ware-out")
2222 35 @Outbound("stock-level-ord-out")
2223 36 @Transactional(type = R)
2224 37 public StockLevelOrdOut processStockLevel(StockLevelWareOut in) {
2225 38     int[] orderIds = IntStream.range(in.next_o_id - 20,
2226 39         in.next_o_id).toArray();
2227 40     int[] itemIds = this.orderLineDB.getAllItemsByOrderIds(orderIds,
2228 41         in.d_id, in.w_id);
2229 42     return new StockLevelOrdOut(in.w_id, itemIds);
2230 43 }
2231 44 @Inbound(values = "delivery-in")
2232 45 @Outbound("delivery-out")
2233 46 @Transactional(type = RW)
2234 47 @PartitionBy(clazz = DeliveryIn.class, method = "getId")
2235 48 public DeliveryOut processDelivery(DeliveryIn in) {
2236 49     int no_o_id;
2237 50     int[] customerIds = new int[10];
2238 51     float[] amounts = new float[10];
2239 52
2240 53
2241 54
2242 55
2243 56
2244 57
2245 58
2246 59
2247 60
2248 61
2249 62
2250 63
2251 64
2252 65
2253 66
2254 67
2255 68
2256 69
2257 70
2258 71
2259 72
2260 73
2261 74
2262 75

```

```

50     for(int d_id = 1; d_id <= TPCCConstants.NUM_DIST_PER_WARE;
51         d_id++) {
52         NewOrder newOrder = newOrder =
53             this.newOrderDB.getFirstNewOrder(d_id, w_id);
54         this.newOrderDB.delete(newOrder);
55
56         no_o_id = newOrder.no_o_id;
57         Order order = this.orderDB.lookupByKey(new
58             Order.OrderId(no_o_id, d_id, in.w_id));
59         order.o_carrier_id = in.carrier_id;
60         this.orderDB.update(order);
61
62         Date date = new Date();
63         List<OrderLine> orderLines =
64             this.orderLineDB.getAllByOrderId(no_o_id, d_id,
65             in.w_id);
66
67         float ol_amount = 0;
68
69         for(OrderLine orderLine : orderLines) {
70             orderLine.ol_delivery_d = date;
71             ol_amount += orderLine.ol_amount;
72         }
73
74         this.orderLineDB.updateAll(orderLines);
75
76         customerIds[d_id - 1] = order.o_c_id;
77         amounts[d_id - 1] = ol_amount;
78     }
79     return new DeliveryOut(in.w_id, customerIds, amounts);
80 }

```

Listing 6: DoMS Order

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009